EXERCISE 11.7

Implement the Kleisli composition function `compose`.

We can now state the associative law for monads in a much more symmetric way:

```
compose(compose(f, g), h) == compose(f, compose(g, h))
```

EXERCISE 11.8

Hard: Implement `flatMap` in terms of `compose`. It seems that we've found another minimal set of monad combinators: `compose` and `unit`.

EXERCISE 11.9

Show that the two formulations of the associative law, the one in terms of `flatMap` and the one in terms of `compose`, are equivalent.

11.4.3 The identity laws

The other monad law is now pretty easy to see. Just like `zero` was an *identity element* for `append` in a monoid, there's an identity element for `compose` in a monad. Indeed, that's exactly what `unit` is, and that's why we chose this name for this operation:[8]

```
def unit[A](a: => A): F[A]
```

This function has the right type to be passed as an argument to `compose`.[9] The effect should be that anything composed with `unit` is that same thing. This usually takes the form of two laws, *left identity* and *right identity*:

```
compose(f, unit) == f
compose(unit, f) == f
```

We can also state these laws in terms of `flatMap`, but they're less clear that way:

```
flatMap(x)(unit) == x
flatMap(unit(y))(f) == f(y)
```

[8] The name *unit* is often used in mathematics to mean an identity for some operation.

[9] Not quite, since it takes a non-strict `A` to `F[A]` (it's an `(=> A) => F[A]`), and in Scala this type is different from an ordinary `A => F[A]`. We'll ignore this distinction for now, though.